Notes on Machine Learning

Course of Machine Learning

Master Degree in Computer Science

University of Rome "Tor Vergata"

Giorgio Gambosi

Academic Year: 2025-2026

Chapter 1

Support Vector Machines

Idea

Support Vector Machines (SVMs) address the problem of binary classification by searching for a hyperplane that separates two classes in a given feature space. The central idea, however, is not merely to find *any* separating hyperplane, but to identify one that is optimal according to a precise and geometrically meaningful criterion: the maximization of the margin. Intuitively, among all possible decision boundaries that correctly classify the training data, we prefer the one that leaves the largest possible "safety buffer" between the two classes.

This perspective immediately highlights two fundamental difficulties that arise in practice. First, real data are often not perfectly linearly separable, so insisting on flawless separation may be unrealistic or even harmful. Second, even when linear separation fails in the original feature space, it may become possible after a suitable transformation of the data. These considerations motivate two complementary extensions of the basic idea. On the one hand, we can relax the notion of separation by allowing some violations of the margin constraints, leading to the concept of *soft margins*. On the other hand, we can enrich the feature space, typically through nonlinear mappings implemented via kernel functions, so that a linear separator in the new space corresponds to a nonlinear decision boundary in the original one.

Before formalizing these ideas, it is useful to recall how linear classifiers produce predictions and how these predictions are connected to geometry.

Classification Scores and Decision Boundaries

Most classification algorithms do not directly output a class label, but rather a numerical score that quantifies how strongly a data point is associated with one class or the other. The final label is then obtained by comparing this score with a fixed threshold. In the case of linear classifiers, this score is an affine function of the input features:

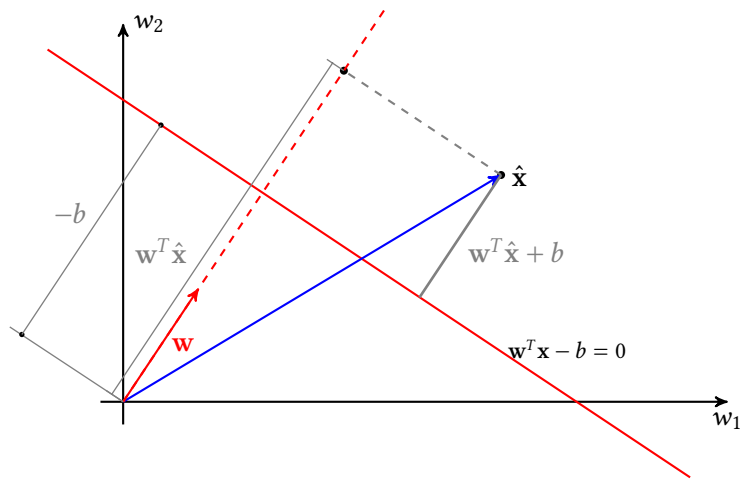
$$\text{score}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b.$$

The vector $\mathbf{w}$ determines the orientation of the decision boundary, while the scalar b controls its position. Different learning algorithms interpret this score in different ways. For instance, logistic regression maps the score through a sigmoid function to produce an estimate of the conditional class probability,

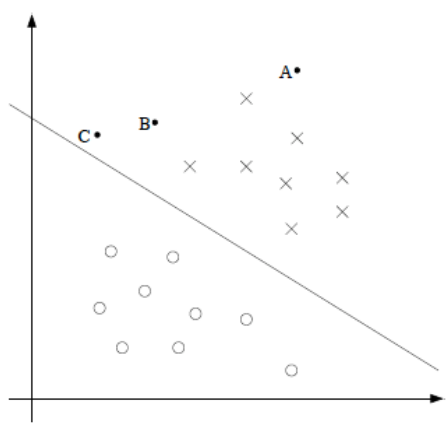
$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) \in [0, 1],$$

and assigns the label 1 whenever this probability exceeds 0.5. Other methods, such as the perceptron or the hard-margin SVM, use only the sign of the score and do not attempt to estimate probabilities.

Beyond its algorithmic role, the score has a clear geometric interpretation. The set of points satisfying $\mathbf{w}^T\mathbf{x} + b = 0$ forms a hyperplane, known as the *decision boundary*. The value of $\mathbf{w}^T\mathbf{x} + b$ is proportional to the signed distance of $\mathbf{x}$ from this hyperplane: its sign indicates on which side of the boundary the point lies, while its magnitude reflects how far the point is from the boundary. Points that are far away are classified with high confidence, whereas points close to the hyperplane are inherently ambiguous.



This geometric viewpoint is illustrated further by comparing points at different distances from the decision boundary. In the figure below, point A lies far from the separating hyperplane and is therefore classified with high confidence, whereas point C, being very close to the boundary, corresponds to a much more uncertain decision. Such observations naturally suggest that, all else being equal, a classifier that places training points far from the decision boundary should generalize better to unseen data.



Margin: Formal Definition

To make these ideas precise, consider a binary classification problem with labels $y \in \{-1, 1\}$, where $y = 1$ denotes class $\mathcal{C}_1$ and $y = -1$ denotes class $\mathcal{C}_0$. We restrict our attention to linear classifiers of the form

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b),$$

which return only a class label and no probabilistic information.

For a training example $(\mathbf{x}_i, t_i)$, with true label $t_i \in \{-1, 1\}$, we define the **functional margin** as

$$\overline{\gamma}_i = t_i(\mathbf{w}^T\mathbf{x}_i + b).$$

This quantity is positive if and only if the example is correctly classified, and its magnitude increases as the point moves away from the decision boundary in the correct direction. From this point of view, the functional margin can be interpreted as a signed confidence measure associated with the prediction on $\mathbf{x}_i$.

Extending this notion to the entire training set $\mathcal{T} = \{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)\}$, we define the functional margin of the classifier as the minimum functional margin over all examples,

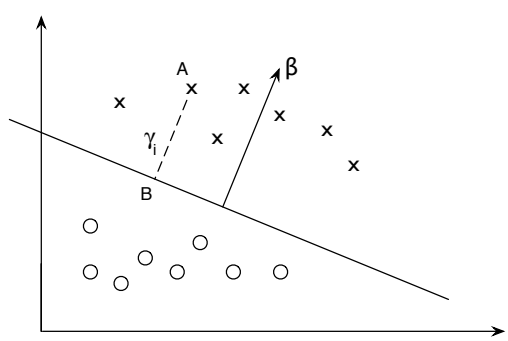
$$\overline{\gamma} = \min i\overline{\gamma}_i.$$

Maximizing this quantity would seem to encourage confident predictions on all training points. However, the functional margin suffers from a fundamental limitation: it is not invariant under rescaling of the parameters. Multiplying $(\mathbf{w}, b)$ by any positive constant α leaves the classifier unchanged, but multiplies $\overline{\gamma}_i$ by α. As a result, the functional margin alone cannot serve as an intrinsic measure of geometric separation.

This issue is resolved by introducing the **geometric margin**, which directly measures distances in the input space. For a single example, the geometric margin is defined as

$$\gamma_i = t_i\left(\frac{\mathbf{w}^T}{\|\mathbf{w}\|}\mathbf{x}_i + \frac{b}{\|\mathbf{w}\|}\right) = \frac{\overline{\gamma}_i}{\|\mathbf{w}\|},$$

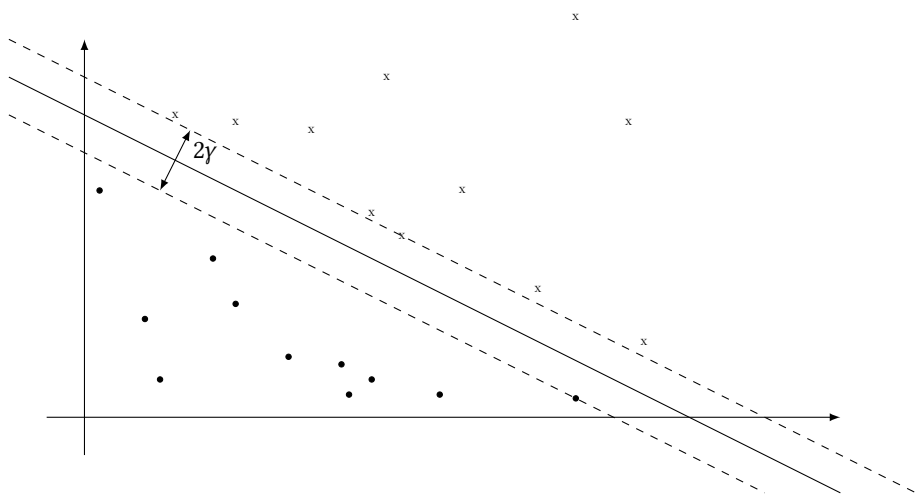
where $\|\mathbf{w}\|$ denotes the Euclidean norm of $\mathbf{w}$. By construction, γ_i is exactly the signed Euclidean distance from $\mathbf{x}_i$ to the separating hyperplane. Importantly, this quantity is invariant under rescaling of the parameters, and thus captures a true geometric property of the classifier.



The geometric margin of the entire training set is then defined as

$$\gamma = \min i\gamma_i.$$

Geometrically, γ corresponds to half the width of the largest slab, centered on the decision hyperplane, that contains no training points. The two hyperplanes delimiting this slab are parallel to the decision boundary and are located at distance γ from it. These are known as the **maximum margin hyperplanes**. Training points that lie exactly on these boundaries play a crucial role and are referred to as **support vectors**.



Interpreting Margin Constraints

The value of $t(\mathbf{w}^T\mathbf{x}+b)$ provides a detailed description of the position of a point relative to the decision boundary and the margin strip. Values larger than 1 correspond to correctly classified points that lie outside the margin, indicating high confidence predictions. When the value is exactly 1, the point lies on the margin boundary and is a support vector. Values between 0 and 1 indicate correctly classified points that lie inside the margin, hence with lower confidence. Negative values correspond to misclassified points, with the magnitude again reflecting how far the point lies on the wrong side of the boundary. This continuum of situations is essential for understanding both hard- and soft-margin formulations.

Optimal Margin Classifiers

Armed with these definitions, we can now formalize the problem of finding an optimal separating hyperplane. When the training data are linearly separable, the goal is to find the classifier that maximizes the geometric margin. From a statistical perspective, a large margin acts as a form of regularization, reducing the sensitivity of the classifier to noise and small perturbations in the data, and thereby improving generalization performance.

The Optimization Problem

For a linearly separable training set, the problem can be stated as

$$\begin{aligned} &\max \mathbf{w}, b\ \gamma \\ &\text{subject to} \quad \gamma_i = \frac{t_i}{\|\mathbf{w}\|}(\mathbf{w}^T\mathbf{x}_i + b) \geq \gamma, \quad i = 1, \dots, n. \end{aligned}$$

Multiplying both sides of the constraints by $\|\mathbf{w}\|$ yields an equivalent formulation,

$$\begin{aligned} &\max \mathbf{w}, b\ \gamma \\ &\text{subject to} \quad t_i(\mathbf{w}^T\mathbf{x}_i + b) \geq \gamma\|\mathbf{w}\|, \quad i = 1, \dots, n. \end{aligned}$$

The key observation is that the decision boundary is invariant under positive rescaling of $(\mathbf{w}, b)$. We can therefore fix this degree of freedom by imposing the normalization condition

$$\min i t_i(\mathbf{w}^T\mathbf{x}_i + b) = 1.$$

Under this constraint, the margin becomes $\gamma = 1/\|\mathbf{w}\|$, and maximizing the margin is equivalent to minimizing the norm of $\mathbf{w}$. This leads to the canonical hard-margin SVM optimization problem,

$$\begin{aligned} &\min \mathbf{w}, b\ \frac{1}{2}\|\mathbf{w}\|^2 \\ &\text{subject to} \quad t_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Properties of the Optimization Problem

The resulting optimization problem is a convex quadratic program. The objective function is strictly convex in $\mathbf{w}$, and the constraints define a convex feasible region given by the intersection of half-spaces. As a consequence, the problem admits a unique global optimum, which can be found efficiently using well-established optimization techniques.

At the optimum, not all constraints are equally important. Points for which

$$t_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$$

are said to be **active** and correspond precisely to the support vectors. These points alone determine the position of the optimal hyperplane. All other training examples, which satisfy the constraints strictly, could be removed without affecting the solution. This sparsity property is one of the most appealing features of SVMs.

Maximum Margin Classifiers and VC Dimension

The preference for large margins is not only geometrically intuitive but also deeply rooted in the theory of statistical learning. In particular, margin maximization is closely connected to the notion of *capacity control*, which is formalized by the Vapnik–Chervonenkis (VC) dimension. The VC dimension provides a way to quantify how complex or expressive a hypothesis class is, that is, how flexibly it can adapt to different labeling patterns of the data.

Recall that the VC dimension of a hypothesis class $\mathcal{H}$ is defined as the largest integer m such that there exists a set of m points that can be *shattered* by $\mathcal{H}$, meaning that $\mathcal{H}$ can realize all 2^m possible binary labelings of those points. A large VC dimension indicates a rich hypothesis class that can fit a wide variety of patterns, but this expressiveness comes at a cost: high-capacity models are more prone to overfitting, i.e., fitting not only the underlying structure of the data but also noise and random fluctuations.

For linear classifiers in $\mathbb{R}^d$ without any further restrictions, the VC dimension is $d+1$. This result reflects the fact that, in a d-dimensional space, hyperplanes are highly flexible objects, especially when d is large. From the perspective of classical VC theory, this would suggest that linear classifiers in high dimensions should generalize poorly unless the number of training samples is very large. Yet, Support Vector Machines often generalize remarkably well even in extremely high-dimensional feature spaces, including infinite-dimensional ones induced by kernels. The key to resolving this apparent paradox lies in the role played by the margin.

When we restrict our attention to classifiers that separate the data with a margin of at least γ, we are no longer considering the full class of all hyperplanes. Instead, we are focusing on a much smaller and more constrained subset. Intuitively, requiring a large margin forces the decision boundary to be relatively stable: small perturbations of the input points cannot easily change their classification. This stability reduces the effective flexibility of the classifier, even if the ambient space is high-dimensional.

This intuition can be made precise through margin-based bounds on the VC dimension. Suppose that all data points lie within a ball of radius R in $\mathbb{R}^d$. One can show that the VC dimension of the class of hyperplanes that separate the data with margin at least γ is bounded by a quantity of order

$$\mathrm{VC} \lesssim \left(\frac{R}{\gamma}\right)^2.$$

Crucially, this bound does *not* depend explicitly on the dimension d of the space. What matters instead is the ratio between the size of the region containing the data (measured by R) and the margin γ. A larger margin directly translates into a smaller upper bound on the VC dimension, meaning that the effective capacity of the hypothesis class is reduced.

This observation explains why maximizing the margin can be interpreted as minimizing an *upper bound* on the VC dimension. While the exact VC dimension of the margin-constrained class may be difficult to compute, the bound $(R/\gamma)^2$ is sufficient to control generalization. In the framework of statistical learning theory, generalization error bounds typically take the form

$$\text{generalization error} \;\leq\; \text{training error} \;+\; \text{complexity term},$$

where the complexity term grows with the VC dimension (or an upper bound on it) and decreases with the number of training samples. By maximizing γ, the SVM reduces the complexity term in these bounds, without necessarily increasing the training error (in the separable case, the training error remains zero).

From this perspective, overfitting can be understood as the consequence of choosing a hypothesis from a class that is too complex relative to the amount of available data. A classifier with a small margin can contort itself to fit the training data very tightly, potentially exploiting accidental alignments or noise. In contrast, a large-margin classifier is constrained to make decisions that are robust to small changes in the data, which prevents it from encoding overly fine-grained distinctions that are unlikely to generalize.

Margin maximization therefore plays a role analogous to regularization. Traditional regularization techniques explicitly penalize model complexity through additional terms in the objective function, such as ℓ_2 or ℓ_1 norms of the parameters. In SVMs, this regularization effect emerges naturally and geometrically: minimizing $\|\mathbf{w}\|^2$ is equivalent to maximizing the margin, which in turn minimizes an upper bound on the VC dimension. The resulting classifier achieves a balance between fitting the data and controlling capacity, leading to strong generalization performance even in high-dimensional or infinite-dimensional feature spaces.

In summary, the theoretical link between maximum margin classifiers and VC dimension provides a rigorous explanation for the empirical success of SVMs. By maximizing the geometric margin, SVMs implicitly control the capacity of the hypothesis class, reduce the risk of overfitting, and ensure that good performance on the training set is more likely to translate into good performance on unseen data.

Duality in Convex Optimization

A fundamental property of convex optimization problems is the existence of an equivalent *dual* formulation. The dual problem provides an alternative perspective on the same optimization task and, under certain conditions, yields computational advantages. This section develops the concept of duality rigorously.

Problem Setup

Consider the constrained optimization problem:

$$\min \mathbf{x} \in \Omega f(\mathbf{x})$$

where the feasible region Ω is defined by inequality constraints:

$$g_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \dots, k$$

Throughout this section, we assume that $f(\mathbf{x})$ and all $g_i(\mathbf{x})$ are convex functions, and that Ω is a convex set. We refer to this as the **primal problem**.

The Lagrangian

The notion of a **Lagrangian** arises naturally when we try to formalize the idea of optimization in the presence of constraints. In many practical problems, we are not free to choose the variables $\mathbf{x}$ arbitrarily: instead, admissible

solutions must satisfy a collection of constraints that encode physical limitations, resource bounds, consistency requirements, or modeling assumptions. The central conceptual difficulty is that constrained optimization couples two competing objectives: on the one hand, we wish to minimize (or maximize) a given cost function; on the other hand, we must enforce that the solution remains within the feasible region defined by the constraints.

A useful way to think about this situation is to imagine that constraint violations come with a *cost*. Rather than treating constraints as rigid, binary conditions (either satisfied or violated), we temporarily allow violations but penalize them in a controlled way. The Lagrangian provides exactly this mechanism: it transforms a constrained optimization problem into an unconstrained one by augmenting the original objective with penalty terms that measure how much each constraint is violated.

More precisely, consider an optimization problem in which we aim to minimize a function $f(\mathbf{x})$ subject to inequality constraints

$$g_i(\mathbf{x}) \leq 0, \qquad i = 1, \dots, k.$$

The Lagrangian associated with this problem is defined as

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{i=1}^{k} \lambda_i g_i(\mathbf{x}),$$

where the vector $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_k)$ collects the **Lagrangian multipliers**, also known as **dual variables**. Each multiplier λ_i is associated with a specific constraint $g_i(\mathbf{x}) \leq 0$ and controls how strongly violations of that constraint are penalized.

This construction admits a clear and intuitive interpretation. If, for a given $\mathbf{x}$, a constraint $g_i(\mathbf{x})$ is satisfied, then $g_i(\mathbf{x}) \leq 0$ and the corresponding term $\lambda_i g_i(\mathbf{x})$ does not increase the value of the Lagrangian. If instead the constraint is violated, $g_i(\mathbf{x}) > 0$, the penalty term becomes positive and increases $L(\mathbf{x}, \boldsymbol{\lambda})$ in proportion both to the amount of violation and to the multiplier λ_i. In this sense, the multipliers play the role of adjustable weights that determine how costly it is to violate each constraint.

From a geometric perspective, the Lagrangian blends information about the objective function and the constraints into a single scalar quantity. Minimizing $L(\mathbf{x}, \boldsymbol{\lambda})$ with respect to $\mathbf{x}$ for fixed $\boldsymbol{\lambda}$ corresponds to searching for solutions that strike a balance between reducing the objective $f(\mathbf{x})$ and avoiding constraint violations. At the same time, varying $\boldsymbol{\lambda}$ changes this balance, effectively reshaping the optimization landscape so as to enforce feasibility.

Each multiplier λ_i can also be viewed as a *shadow price*: it quantifies how much the optimal value of the objective would improve if the corresponding constraint were relaxed slightly. In this sense, the dual variables encode the sensitivity of the optimal solution to the constraints and provide valuable insight beyond the primal solution $\mathbf{x}$ itself.

Weak Duality

Starting from the Lagrangian, duality can be introduced in a particularly intuitive way by thinking in terms of *two players* and *two complementary viewpoints* on the same constrained optimization problem.

Suppose we wish to minimize an objective $f(\mathbf{x})$ over a feasible set defined by constraints. In the **primal** viewpoint, $\mathbf{x}$ is the decision variable and feasibility is essential: we want the best value of $f(\mathbf{x})$ among admissible solutions. The Lagrangian allows us to encode constraints by augmenting the objective with weighted violations. This immediately suggests an adversarial interpretation: if $\mathbf{x}$ violates a constraint, a multiplier λ_i can be increased to penalize that violation, making the choice of $\mathbf{x}$ unattractive. In other words, the multipliers can be used to *enforce feasibility* indirectly.

This leads naturally to the following minimax expression. For a fixed $\mathbf{x}$, consider maximizing the Lagrangian over multipliers $\boldsymbol{\lambda}$ with $\lambda_i \geq 0$. If $\mathbf{x}$ is feasible, then each $g_i(\mathbf{x}) \leq 0$ and increasing λ_i cannot increase $L(\mathbf{x}, \boldsymbol{\lambda})$, so the maximization does not "punish" $\mathbf{x}$. If $\mathbf{x}$ is infeasible and some constraint has $g_i(\mathbf{x}) > 0$, then the maximization drives λ_i upward and makes $L(\mathbf{x}, \boldsymbol{\lambda})$ arbitrarily large. Hence the maximization over $\boldsymbol{\lambda}$ acts as a mechanism that assigns an infinite cost to infeasible points and leaves feasible points essentially unchanged. Minimizing over $\mathbf{x}$ after this "penalty maximization" reproduces the original constrained problem. This is the primal formulation written in terms of the Lagrangian:

$$P^* = \min \mathbf{x} \in \Omega f(\mathbf{x}) = \min \mathbf{x} \in \Omega \max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 L(\mathbf{x}, \boldsymbol{\lambda}).$$

Here, Ω denotes the domain of $\mathbf{x}$ (for example $\mathbb{R}^d$ or another set encoding basic restrictions), while the constraints are enforced through the maximization over $\boldsymbol{\lambda}$.

The **dual** viewpoint reverses the order of these operations. Instead of choosing $\mathbf{x}$ first, we fix multipliers $\boldsymbol{\lambda}$ and ask: *what is the smallest value of the Lagrangian we can obtain over* $\mathbf{x}$*?* This gives rise to the **dual function**

$$q(\boldsymbol{\lambda}) = \min \mathbf{x} \in \Omega L(\mathbf{x}, \boldsymbol{\lambda}),$$

which assigns to each set of multipliers the best (i.e., lowest) value of the penalized objective that can be achieved. Importantly, for any $\boldsymbol{\lambda} \succeq 0$ this quantity provides a *lower bound* on the primal optimum P^* (for minimization problems). The intuition is the following: when $\mathbf{x}$ is feasible, $g_i(\mathbf{x}) \leq 0$ and thus $\sum_i \lambda_i g_i(\mathbf{x}) \leq 0$, meaning that

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_i \lambda_i g_i(\mathbf{x}) \leq f(\mathbf{x}).$$

Taking the minimum over $\mathbf{x}$ on the left-hand side cannot exceed the minimum of $f(\mathbf{x})$ over feasible points, so $q(\boldsymbol{\lambda}) \leq P^*$ for all admissible $\boldsymbol{\lambda}$. Therefore, by searching over multipliers we can try to make this bound as tight as possible.

This idea defines the dual optimization problem:

$$D^* = \max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 \min \mathbf{x} \in \Omega L(\mathbf{x}, \boldsymbol{\lambda}) = \max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 q(\boldsymbol{\lambda}).$$

In words, the dual problem chooses the multipliers so that the *best lower bound* produced by the Lagrangian is as large as possible.

Relationship between primal and dual solutions.

The connection between primal and dual solutions can be understood most clearly by interpreting the Lagrangian as a device that simultaneously evaluates the objective function and measures constraint violations. From this perspective, primal and dual variables play complementary roles: primal variables propose candidate solutions, while dual variables assess and penalize their feasibility.

A central consequence of this interaction is the following fundamental principle:

Any feasible dual solution provides a lower bound on the optimal value of the primal problem.

This statement is the essence of **weak duality**. To understand why it holds, it is instructive to analyze how the Lagrangian behaves depending on whether a candidate primal point $\mathbf{x}$ satisfies the constraints or not.

Consider first a point $\mathbf{x}$ that is **feasible**, meaning that all inequality constraints are satisfied:

$$g_i(\mathbf{x}) \leq 0 \qquad \forall i.$$

For any choice of multipliers $\lambda_i \geq 0$, each product $\lambda_i g_i(\mathbf{x})$ is non-positive. As a consequence, the Lagrangian satisfies

$$L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_i \lambda_i g_i(\mathbf{x}) \leq f(\mathbf{x}).$$

If we now maximize $L(\mathbf{x}, \boldsymbol{\lambda})$ with respect to $\boldsymbol{\lambda}$, the best strategy is to avoid penalizing the objective at all, which is achieved by setting $\lambda_i = 0$ for all i. Hence, for feasible $\mathbf{x}$,

$$\max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 L(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}).$$

This shows that, on feasible points, the Lagrangian reproduces exactly the original objective function.

The situation changes dramatically when $\mathbf{x}$ is **infeasible**. Suppose that at least one constraint is violated, so that there exists an index i such that

$$g_i(\mathbf{x}) > 0.$$

Since the corresponding multiplier λ_i is constrained only to be non-negative, we can increase it arbitrarily. As $\lambda_i \to +\infty$, the penalty term $\lambda_i g_i(\mathbf{x})$ grows without bound, and therefore the Lagrangian diverges:

$$\max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 L(\mathbf{x}, \boldsymbol{\lambda}) = +\infty.$$

Thus, infeasible points are assigned an infinite cost once the maximization over multipliers is taken into account.

Combining these two observations leads to a key insight. The operation

$$\max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 L(\mathbf{x}, \boldsymbol{\lambda})$$

acts as a *filter*: it leaves the objective value unchanged on feasible points and discards infeasible ones by assigning them infinite cost. Consequently, minimizing this quantity over $\mathbf{x}$ is equivalent to minimizing $f(\mathbf{x})$ over the feasible set:

$$\min \mathbf{x} \max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 L(\mathbf{x}, \boldsymbol{\lambda}) = \min \mathbf{x} \in \Omega f(\mathbf{x}) = P^*.$$

This expression recovers the primal problem from the Lagrangian formulation.

The dual problem arises when the order of minimization and maximization is reversed. Instead of first enforcing feasibility via the maximization over $\boldsymbol{\lambda}$, we first minimize the Lagrangian over $\mathbf{x}$ for a fixed choice of multipliers. For any $\boldsymbol{\lambda} \succeq 0$, this produces a value

$$\min \mathbf{x} L(\mathbf{x}, \boldsymbol{\lambda}),$$

which, by construction, cannot exceed the primal optimum. Maximizing this quantity over $\boldsymbol{\lambda}$ therefore yields the tightest possible lower bound obtainable through the Lagrangian:

$$\max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 \min \mathbf{x} L(\mathbf{x}, \boldsymbol{\lambda}) \leq P^*.$$

This inequality is precisely the statement of **weak duality**.

Strong Duality

Weak duality tells us that the dual problem always provides a lower bound on the primal optimum. In general, however, this bound may be strict: the best value attainable by the dual problem may remain strictly smaller than the optimal value of the primal problem. The difference

$$P^* - D^*$$

is called the *duality gap* and quantifies how far the two formulations are from each other.

A remarkable and highly nontrivial fact is that, for a broad and important class of optimization problems, this gap vanishes entirely. In particular, for **convex optimization problems**—including quadratic programs such as the Support Vector Machine (SVM)—the primal and dual formulations are not merely related by an inequality, but are in fact equivalent in terms of their optimal values. This property is known as **strong duality**.

Formally, strong duality means that the optimal value of the primal problem coincides with the optimal value of the dual problem:

$$D^* = \max \boldsymbol{\lambda} \ : \ \lambda_i \geq 0 \min \mathbf{x} L(\mathbf{x}, \boldsymbol{\lambda}) = \min \mathbf{x} \in \Omega f(\mathbf{x}) = P^*.$$

In other words, exchanging the order of minimization over the primal variables and maximization over the dual variables does not change the final result.

From a practical standpoint, strong duality has profound consequences. Since the dual problem often has a simpler structure than the primal—fewer variables, simpler constraints, or better numerical properties—it can be advantageous to solve the dual problem instead. When strong duality holds, solving the dual is not merely an approximation: it yields the exact optimal value of the primal problem.

Karush–Kuhn–Tucker (KKT) conditions

The Karush–Kuhn–Tucker (KKT) conditions provide a precise and powerful characterization of optimal solutions to constrained optimization problems. Conceptually, they can be seen as the natural generalization of the familiar condition "the gradient must vanish at an optimum" from unconstrained optimization to the case in which constraints restrict the feasible region.

For a convex optimization problem with inequality constraints $g_i(\mathbf{x}) \leq 0$, the KKT conditions are *necessary and sufficient* for global optimality. That is, a point $(\mathbf{x}^*, \boldsymbol{\lambda}^*)$ is primal–dual optimal if and only if it satisfies the following conditions.

1. **Stationarity (first-order optimality).**
   The gradient of the Lagrangian with respect to the primal variables must vanish at the optimum:

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda})\big|_{\mathbf{x}=\mathbf{x}^*,\, \boldsymbol{\lambda}=\boldsymbol{\lambda}^*} = \nabla f(\mathbf{x}^*) + \sum_{i=1}^{k} \lambda_i^* \nabla g_i(\mathbf{x}^*) = \mathbf{0}.$$

   This condition generalizes the unconstrained requirement $\nabla f(\mathbf{x}^*) = \mathbf{0}$. The additional terms $\lambda_i^* \nabla g_i(\mathbf{x}^*)$ represent the forces exerted by the active constraints. At the optimum, these forces exactly balance the gradient of the objective, preventing further descent in feasible directions.
2. **Dual feasibility.**
   The Lagrange multipliers must be nonnegative:

$$\lambda_i^* \geq 0 \qquad i = 1, \dots, k.$$

   This reflects the fact that constraint violations can only be penalized, never rewarded. A negative multiplier would encourage violating a constraint, which is incompatible with the original constrained problem.
3. **Primal feasibility.**
   The optimal primal solution must satisfy all constraints:

$$g_i(\mathbf{x}^*) \leq 0 \qquad i = 1, \dots, k.$$

   This is simply the requirement that $\mathbf{x}^*$ belongs to the feasible set of the original problem.
4. **Complementary slackness.**
   For each constraint, the product of the multiplier and the constraint value must vanish:

$$\lambda_i^* \, g_i(\mathbf{x}^*) = 0 \qquad i = 1, \dots, k.$$

Interpretation of complementary slackness.

Complementary slackness provides one of the most intuitive and informative insights into constrained optimization. It states that, at the optimum, each constraint must be in exactly one of two regimes:

- the constraint is **active**, meaning $g_i(\mathbf{x}^*) = 0$, in which case the associated multiplier λ_i^* may be strictly positive and contributes to the stationarity condition;
- the constraint is **inactive**, meaning $g_i(\mathbf{x}^*) < 0$, in which case the multiplier must vanish, $\lambda_i^* = 0$, and the constraint plays no role in determining the optimum.

In intuitive terms, only constraints that are "tight" at the solution influence the optimal point. Constraints that are strictly satisfied are irrelevant locally: removing them would not change the solution.

The KKT conditions above characterize stationary points of the constrained problem. To ensure that such a point is indeed a minimum (and not, for example, a saddle point), a second-order condition is required. Specifically, the Hessian of the Lagrangian with respect to the primal variables,

$$H_{\mathbf{x}} = \nabla_{\mathbf{x}}^2 L(\mathbf{x}^*, \boldsymbol{\lambda}^*),$$

must be positive semidefinite on the space of feasible directions.

For convex optimization problems, this requirement is however automatically satisfied: the objective function is convex, the constraint functions are convex, and therefore the Lagrangian is convex in $\mathbf{x}$. As a result, any point satisfying the KKT conditions is guaranteed to be a global minimum.

The Dual Problem in Standard Form

Given the KKT conditions, the dual problem can be expressed explicitly. Define the dual function:

$$D(\boldsymbol{\lambda}) = \min \mathbf{x} L(\mathbf{x}, \boldsymbol{\lambda})$$

The dual problem is:

$$\begin{aligned} &\max \boldsymbol{\lambda}\, D(\boldsymbol{\lambda}) \\ &\text{subject to} \quad \lambda_i \geq 0 \quad \text{for } i = 1, \dots, k \end{aligned}$$

The dual problem is always concave in $\boldsymbol{\lambda}$ (even if the primal is not convex), which makes it easier to solve in some cases. Combined with strong duality, solving the dual can yield the primal optimum.

This formulation is not an ad hoc construction, but follows directly and systematically from the considerations developed in the previous sections. In particular, it is the natural consequence of three key observations:

- the Lagrangian provides, for each fixed $\boldsymbol{\lambda} \succeq 0$, a lower bound on the primal objective;
- minimizing the Lagrangian over $\mathbf{x}$ extracts the *best* such bound associated with those multipliers;
- maximizing over $\boldsymbol{\lambda}$ selects the tightest bound among all admissible penalties.

Practical Significance for SVMs

The duality framework is especially valuable for Support Vector Machines:

- The primal SVM problem involves optimizing the weight vector $\mathbf{w}$ in high-dimensional feature space. In contrast, the dual problem involves only the Lagrangian multipliers λ_i, one per training example.
- The dual formulation enables the use of *kernel methods*, which allow SVMs to work implicitly in very high-dimensional (even infinite-dimensional) feature spaces without explicitly computing feature vectors.
- Complementary slackness reveals that only a small subset of training points (the support vectors) have $\lambda_i^* > 0$ and thus contribute to the final classifier. This sparsity makes SVMs computationally efficient and provides interpretability.
- The dual problem structure allows efficient algorithms (such as Sequential Minimal Optimization) to solve large-scale SVM training problems.

Practical Significance for SVMs

The duality framework is especially valuable for Support Vector Machines, both from a theoretical and a practical standpoint:

- The primal SVM problem optimizes the weight vector $\mathbf{w}$ (and bias b) in feature space, whose dimension may be very large. By contrast, the dual problem involves only one variable λ_i per training example, shifting the complexity from the feature dimension to the number of data points.
- The dual formulation expresses the optimization problem entirely in terms of inner products between training examples. This structural feature is what enables the use of **kernel methods**, allowing SVMs to operate implicitly in high-dimensional or even infinite-dimensional feature spaces.
- Complementary slackness reveals that only a subset of training points have nonzero optimal multipliers λ_i^*. These points, called **support vectors**, are the only ones that influence the final classifier. This sparsity improves computational efficiency and enhances interpretability.
- The structure of the dual problem admits specialized and highly efficient optimization algorithms, such as Sequential Minimal Optimization (SMO), which are well suited for large-scale learning problems.

Application to SVM: The Dual Problem

This section derives the dual formulation of the hard-margin SVM optimization problem in a rigorous manner, starting from the primal problem and applying the general duality and KKT framework developed earlier. The derivation highlights how the structure of the dual naturally emerges from the constrained formulation and why it is particularly well suited for kernel-based learning.

Primal Problem Setup

Recall the primal SVM optimization problem:

$$\begin{aligned} &\min \mathbf{w}, b \ \frac{1}{2}\|\mathbf{w}\|^2 \\ &\text{subject to} \quad t_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \quad \text{for } i = 1, \dots, n. \end{aligned}$$

This is a convex optimization problem with linear inequality constraints. In the notation of the general framework:

- The objective function is
$$f(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2,$$
which is strictly convex in $\mathbf{w}$.
- The constraint functions are
$$g_i(\mathbf{w}, b) = -\big(t_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big) \leq 0, \quad i = 1, \dots, n,$$
which are affine functions of $(\mathbf{w}, b)$.
- The feasible region is a convex polyhedron defined by these linear constraints.

Since the problem is convex and Slater's condition is satisfied whenever the data are linearly separable, strong duality holds.

Formulating the Lagrangian

Introducing a nonnegative Lagrange multiplier $\lambda_i \geq 0$ for each constraint, the Lagrangian is

$$\begin{aligned} L(\mathbf{w}, b, \boldsymbol{\lambda}) &= \frac{1}{2}\|\mathbf{w}\|^2 + \sum_{i=1}^{n} \lambda_i \, g_i(\mathbf{w}, b) \\ &= \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \lambda_i \big(t_i(\mathbf{w}^T\mathbf{x}_i + b) - 1\big). \end{aligned}$$

Expanding the terms yields

$$L(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{n} \lambda_i t_i \mathbf{w}^T\mathbf{x}_i - b\sum_{i=1}^{n} \lambda_i t_i + \sum_{i=1}^{n} \lambda_i,$$

with $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_n) \succeq 0$.

Deriving the Dual via Stationarity

The dual function is obtained by minimizing the Lagrangian with respect to the primal variables $(\mathbf{w}, b)$ for fixed multipliers $\boldsymbol{\lambda}$. Because the Lagrangian is convex in $(\mathbf{w}, b)$, this minimization is characterized by the stationarity conditions.

Stationarity with respect to $\mathbf{w}$.
Setting the gradient of L with respect to $\mathbf{w}$ equal to zero gives

$$\nabla_{\mathbf{w}} L = \mathbf{w} - \sum_{i=1}^{n} \lambda_i t_i \mathbf{x}_i = \mathbf{0},$$

which implies

$$\mathbf{w}^* = \sum_{i=1}^{n} \lambda_i t_i \mathbf{x}_i.$$

Stationarity with respect to b.
Taking the derivative with respect to b yields

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \lambda_i t_i = 0,$$

which leads to the equality constraint

$$\sum_{i=1}^{n} \lambda_i t_i = 0.$$

Key insight. At optimality, the weight vector $\mathbf{w}^*$ lies in the span of the training examples. This representation, derived directly from the KKT stationarity conditions, is the mathematical foundation of the support vector expansion.

Explicit Form of the Dual Function

Substituting the optimal expressions for $\mathbf{w}^*$ and the constraint on $\boldsymbol{\lambda}$ into the Lagrangian eliminates the primal variables. We obtain

$$\begin{aligned} D(\boldsymbol{\lambda}) &= L(\mathbf{w}^*, b^*, \boldsymbol{\lambda}) \\ &= \frac{1}{2}\left(\sum_{i=1}^{n} \lambda_i t_i \mathbf{x}_i\right)^T \left(\sum_{j=1}^{n} \lambda_j t_j \mathbf{x}_j\right) - \sum_{i=1}^{n} \lambda_i t_i \left(\sum_{j=1}^{n} \lambda_j t_j \mathbf{x}_j^T \mathbf{x}_i\right) + \sum_{i=1}^{n} \lambda_i \\ &= \sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j t_i t_j \mathbf{x}_i^T \mathbf{x}_j. \end{aligned}$$

This expression depends only on the dual variables $\boldsymbol{\lambda}$ and the training data.

Standard Dual SVM Problem

The dual SVM problem is therefore:

$$\max \boldsymbol{\lambda} \left(\sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j t_i t_j \mathbf{x}_i^T \mathbf{x}_j \right)$$
$$\text{subject to} \quad \lambda_i \geq 0 \quad \text{for } i = 1, \dots, n,$$
$$\sum_{i=1}^{n} \lambda_i t_i = 0.$$

By strong duality, the optimal value of this problem coincides with that of the primal SVM problem.

Comparison with the primal formulation. The primal problem involves $d + 1$ variables and n constraints, whereas the dual problem involves n variables, one equality constraint, and simple nonnegativity constraints. This shift in structure is central to the effectiveness of SVMs in high-dimensional settings.

Interpretation of the Dual Variables

The KKT complementary slackness condition,

$$\lambda_i^* \left(t_i (\mathbf{w}^{*T} \mathbf{x}_i + b^*) - 1 \right) = 0,$$

implies that:

- if $\lambda_i^* > 0$, then $t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) = 1$ and the point lies exactly on the margin;
- if $t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) > 1$, then $\lambda_i^* = 0$ and the point does not influence the classifier.

Thus, only support vectors contribute to the decision function.

Kernel Formulation

A decisive advantage of the dual formulation is that the data appear only through inner products $\mathbf{x}_i^T \mathbf{x}_j$. This allows us to replace them with a kernel function

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j,$$

or, more generally,

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x}_j),$$

where $\boldsymbol{\phi}$ is an implicit feature map.

The dual problem in kernelized form becomes:

$$\max \boldsymbol{\lambda} \left(\sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j t_i t_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \right)$$
$$\text{subject to} \quad \lambda_i \geq 0 \quad \text{for } i = 1, \dots, n,$$
$$\sum_{i=1}^{n} \lambda_i t_i = 0.$$

This formulation enables SVMs to learn nonlinear decision boundaries while preserving the convexity and strong duality properties of the original optimization problem.

Support Vectors and Complementary Slackness

By the complementary slackness condition (the last KKT condition), for each training example we have

$$\lambda_i^* \left(t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) - 1 \right) = 0.$$

This condition establishes a precise correspondence between the optimal dual variables and the geometric role of the training points:

- If $\lambda_i^* > 0$, then necessarily
$$t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) = 1,$$
meaning that the point $\mathbf{x}_i$ lies exactly on the margin boundary. Such points are called **support vectors**.
- If $\lambda_i^* = 0$, then the constraint is inactive and
$$t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) \geq 1.$$
The point is correctly classified with a margin strictly larger than one and does not influence the optimal separating hyperplane.

Thus, complementary slackness provides a rigorous explanation for the sparsity of the SVM solution: only training points that lie on the margin contribute to defining the classifier. In practice, the number of support vectors is typically much smaller than the total number of training examples n, yielding compact models and efficient prediction.

Recovering Primal Parameters from the Dual Solution

Once the dual optimization problem has been solved and the optimal multipliers

$$\boldsymbol{\lambda}^* = (\lambda_1^*, \dots, \lambda_n^*)$$

have been obtained, the corresponding primal parameters can be recovered directly from the KKT conditions.

Recovery of $\mathbf{w}^*$.
From the stationarity condition with respect to $\mathbf{w}$, we obtain

$$\mathbf{w}^* = \sum_{i=1}^{n} \lambda_i^* t_i \mathbf{x}_i.$$

Only support vectors (those with $\lambda_i^* > 0$) contribute to this sum, confirming that $\mathbf{w}^*$ lies in the span of the support vectors.

Recovery of b^*.
For any support vector $\mathbf{x}_k$ with $\lambda_k^* > 0$, the corresponding constraint is active:

$$t_k(\mathbf{w}^{*T}\mathbf{x}_k + b^*) = 1.$$

Solving for b^* yields

$$\begin{aligned} b^* &= \frac{1}{t_k} - \mathbf{w}^{*T}\mathbf{x}_k = t_k - \mathbf{w}^{*T}\mathbf{x}_k \qquad (\text{since } t_k^2 = 1) \\ &= t_k - \sum_{j=1}^{n} \lambda_j^* t_j \mathbf{x}_j^T \mathbf{x}_k. \end{aligned}$$

In kernel form, this becomes

$$b^* = t_k - \sum_{j=1}^{n} \lambda_j^* t_j \kappa(\mathbf{x}_j, \mathbf{x}_k).$$

Because multiple support vectors may exist, numerical stability is improved by averaging this expression over all support vectors:

$$b^* = \frac{1}{|\mathcal{S}|} \sum_{k \in \mathcal{S}} \left(t_k - \sum_{j \in \mathcal{S}} \lambda_j^* t_j \kappa(\mathbf{x}_j, \mathbf{x}_k) \right),$$

where

$$\mathcal{S} = \{\, i \mid \lambda_i^* > 0 \,\}$$

denotes the set of support vector indices.

Classification Using the Trained SVM

Once the optimal parameters $\mathbf{w}^*$ and b^* have been determined, a new point $\mathbf{x}$ is classified using the decision function

$$y(\mathbf{x}) = \mathbf{w}^{*T}\mathbf{x} + b^*.$$

Substituting the support vector expansion of $\mathbf{w}^*$ gives

$$y(\mathbf{x}) = \sum_{i=1}^{n} \lambda_i^* t_i \mathbf{x}_i^T \mathbf{x} + b^*,$$

and the predicted label is $\mathrm{sign}(y(\mathbf{x}))$.

Using a kernel function, the decision function becomes

$$y(\mathbf{x}) = \sum_{i=1}^{n} \lambda_i^* t_i \kappa(\mathbf{x}_i, \mathbf{x}) + b^*.$$

By sparsity of the solution, this expression simplifies to

$$y(\mathbf{x}) = \sum_{i \in S} \lambda_i^* t_i \kappa(\mathbf{x}_i, \mathbf{x}) + b^*,$$

where the sum runs only over support vectors.

Computational efficiency. The decision function requires kernel evaluations only between the new point and the support vectors, rather than the entire training set. When $|S| \ll n$, prediction is therefore fast and memory-efficient.

Key advantages of this formulation:

- The classifier depends on the data exclusively through kernel evaluations, enabling nonlinear decision boundaries without explicit feature construction.
- The sparsity induced by complementary slackness yields interpretable models and efficient computation at both training and test time.
- The kernel-based dual formulation illustrates how general principles of convex duality translate directly into practical and powerful learning algorithms.

Soft-Margin SVM: Handling Non-Separable Data

The hard-margin SVM formulation developed in the previous section relies on the assumption that the training data are linearly separable in feature space. While this assumption is mathematically convenient and leads to a clean geometric picture, it is rarely satisfied in practice. Real-world data are often noisy, may contain outliers, or may simply not admit a perfect separating hyperplane, even after mapping into a high-dimensional feature space via a kernel.

The soft-margin SVM extends the hard-margin framework in a principled way to cope with such situations. The key idea is to relax the strict separation constraints, allowing controlled violations, while still preserving the core philosophy of margin maximization.

Motivation: Why Hard Margins Fail

Recall that the hard-margin SVM enforces the constraints

$$t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 \quad \text{for all } i = 1, \dots, n.$$

Geometrically, these inequalities require every training point to lie on the correct side of the separating hyperplane and at least at unit distance (in the scaled sense) from it. When the data are not linearly separable, this system of inequalities becomes infeasible: no choice of $(\mathbf{w}, b)$ can satisfy all constraints simultaneously. As a consequence, the hard-margin optimization problem admits no solution and completely breaks down.

From a modeling perspective, this behavior is undesirable. A single mislabeled point or a small amount of noise should not prevent the learning algorithm from producing a reasonable classifier. This motivates a relaxation of the constraints that tolerates violations, but does so in a controlled and quantifiable way.

Slack Variables and Relaxed Constraints

Constraint relaxation is introduced through the use of **slack variables**. For each training example, we associate a nonnegative slack variable $\xi_i \geq 0$ that measures the degree to which the margin constraint is violated. The hard constraint is replaced by

$$t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 - \xi_i \quad \text{for all } i = 1, \dots, n.$$

The role of ξ_i can be understood geometrically by examining different regimes. If $\xi_i = 0$, the constraint reduces to the hard-margin condition and the point is correctly classified with the required margin. If $0 < \xi_i < 1$, the point is still correctly classified, but it lies inside the margin strip and therefore enjoys a reduced margin. When $\xi_i = 1$, the point lies exactly on the decision boundary. Finally, if $\xi_i > 1$, the point is misclassified and lies on the wrong side of the hyperplane.

In this way, the slack variables provide a continuous measure of constraint violation, allowing the optimization problem to remain feasible even when perfect separation is impossible.

The Soft-Margin Optimization Problem

Introducing slack variables alone is not sufficient: we must also discourage excessive violations. This is achieved by augmenting the objective function with a penalty term that grows with the total slack. The resulting optimization problem is

$$\begin{aligned} &\min \mathbf{w}, b, \boldsymbol{\xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \\ &\text{subject to} \quad t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 - \xi_i \quad \text{for } i = 1, \dots, n, \\ &\qquad\qquad\quad \xi_i \geq 0 \quad \text{for } i = 1, \dots, n, \end{aligned}$$

where $\boldsymbol{\xi} = (\xi_1, \dots, \xi_n)$ and $C > 0$ is a regularization parameter.

The structure of the objective reflects a trade-off between two competing goals. The term $\frac{1}{2}\|\mathbf{w}\|^2$ favors large margins, as in the hard-margin case. The term $C \sum_i \xi_i$ penalizes violations of the margin constraints. The parameter C controls the relative importance of these objectives. For large values of C, violations are heavily penalized and the solution prioritizes correct classification of the training data, potentially at the expense of a smaller margin. For small values of C, violations are penalized less strongly, leading to wider margins but possibly more misclassifications. In practice, C is treated as a hyperparameter and selected by cross-validation.

Lagrangian Formulation

To derive the dual problem and characterize optimal solutions, we construct the Lagrangian associated with the soft-margin primal problem. Two sets of inequality constraints are present: the relaxed margin constraints and the nonnegativity constraints on the slack variables. Accordingly, we introduce Lagrange multipliers $\lambda_i \geq 0$ for the former and $\alpha_i \geq 0$ for the latter. The Lagrangian is

$$\begin{aligned} L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\lambda}, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \\ - \sum_{i=1}^{n} \lambda_i \left(t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) - 1 + \xi_i \right) \\ - \sum_{i=1}^{n} \alpha_i \xi_i. \end{aligned}$$

This expression combines the objective function with weighted penalties for constraint violations, in direct analogy with the hard-margin case, but now enriched by the presence of slack variables.

KKT Conditions

Optimality of a primal–dual solution is characterized by the Karush–Kuhn–Tucker conditions. Stationarity of the Lagrangian with respect to the primal variables yields

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \mathbf{0}, \\ \frac{\partial L}{\partial b} &= 0, \\ \frac{\partial L}{\partial \xi_i} &= 0 \quad \text{for } i = 1, \dots, n. \end{aligned}$$

These are complemented by primal feasibility of the constraints, dual feasibility of the multipliers, and

complementary slackness conditions:

$$
\begin{aligned}
t_i(\mathbf{w}^{*T}\boldsymbol{\phi}(\mathbf{x}_i) + b^*) - 1 + \xi_i^* &\geq 0, \\
\xi_i^* &\geq 0, \\
\lambda_i^*, \alpha_i^* &\geq 0, \\
\lambda_i^* \left(t_i(\mathbf{w}^{*T}\boldsymbol{\phi}(\mathbf{x}_i) + b^*) - 1 + \xi_i^*\right) &= 0, \\
\alpha_i^* \xi_i^* &= 0.
\end{aligned}
$$

Together, these conditions encode both feasibility and optimal balance between margin maximization and violation penalties.

Deriving the Dual Problem

Evaluating the stationarity conditions explicitly yields

$$
\begin{aligned}
\mathbf{w}^* &= \sum_{i=1}^{n} \lambda_i t_i \boldsymbol{\phi}(\mathbf{x}_i), \\
\sum_{i=1}^{n} \lambda_i t_i &= 0, \\
C - \lambda_i - \alpha_i &= 0.
\end{aligned}
$$

The last relation implies $\lambda_i \leq C$ for all i, since $\alpha_i \geq 0$. Substituting the optimal expressions into the Lagrangian and simplifying as in the hard-margin case leads to the dual problem

$$
\begin{aligned}
&\max \boldsymbol{\lambda} \left(\sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j t_i t_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \right) \\
&\text{subject to} \quad 0 \leq \lambda_i \leq C \quad \text{for } i = 1, \dots, n, \\
&\qquad\qquad \sum_{i=1}^{n} \lambda_i t_i = 0.
\end{aligned}
$$

The only formal difference with respect to the hard-margin dual problem is the upper bound $\lambda_i \leq C$. In the limit $C \to \infty$, this constraint disappears and the hard-margin formulation is recovered.

Interpretation via Complementary Slackness

Complementary slackness provides a detailed picture of the role played by each training example. If $\lambda_i^* = 0$ and $\xi_i^* = 0$, the point lies outside the margin and does not influence the solution. If $0 < \lambda_i^* < C$, then $\xi_i^* = 0$ and the point lies exactly on a margin boundary: these are classical support vectors. If $\lambda_i^* = C$, then $\xi_i^* > 0$ and the

point either lies inside the margin or is misclassified. Such points are also support vectors, but they correspond to margin violations.

Thus, as in the hard-margin case, only support vectors contribute to the expansion of the optimal weight vector

$$\mathbf{w}^* = \sum_{i \in \mathcal{S}} \lambda_i^* t_i \boldsymbol{\phi}(\mathbf{x}_i),$$

where $\mathcal{S}$ denotes the set of indices with $\lambda_i^* > 0$.

Recovering Primal Parameters and Classification

The bias term b^* can be recovered using any support vector with $0 < \lambda_i^* < C$:

$$b^* = t_k - \sum_{j \in \mathcal{S}} \lambda_j^* t_j \kappa(\mathbf{x}_j, \mathbf{x}_k).$$

In practice, averaging this expression over all such support vectors improves numerical stability.

Classification of a new point $\mathbf{x}$ then proceeds as

$$y(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* t_i \kappa(\mathbf{x}_i, \mathbf{x}) + b^*,$$

with predicted label $\operatorname{sign}(y(\mathbf{x}))$.

The soft-margin SVM thus preserves the key structural advantages of the hard-margin formulation—sparsity, kernelization, and convex optimization—while extending its applicability to realistic, noisy, and non-separable datasets.

Extensions of SVM

So far, we have developed Support Vector Machines as a method for binary classification, first in the hard-margin setting and then in the soft-margin setting. In both cases, the central ideas were the same: represent the classifier as a separating hyperplane in a (possibly high-dimensional) feature space, maximize the margin, and exploit the dual formulation to obtain a sparse kernelized decision function.

In many applications, however, the learning task departs from the basic binary setting. Two particularly important generalizations are (i) classification with more than two classes and (ii) prediction of continuous outputs rather than discrete labels. In what follows, we describe how the SVM philosophy extends to these settings while preserving the key structural advantages of the method.

Multiclass Classification

The standard SVM is designed for binary labels $t_i \in \{-1, +1\}$. When the dataset contains $K > 2$ classes, there is no unique "canonical" generalization of the margin constraint, because the notion of separation must now distinguish among several categories. A common and effective strategy is therefore to reduce the multiclass problem to a collection of binary SVM problems, each of which can be solved with exactly the machinery already developed.

One-vs-All (OvA) approach.

The most direct reduction is the **one-vs-all** (or one-vs-rest) scheme. The idea is to build K binary classifiers, one for each class. For a fixed class $k \in \{1, \dots, K\}$, we relabel the training set by declaring class k as the positive class and all other classes as negative. Denoting these induced binary labels by $t_{i,k}$, we have $t_{i,k} = +1$ if the original label of $\mathbf{x}_i$ is k, and $t_{i,k} = -1$ otherwise.

For each class k, we then train a standard (soft-margin) SVM classifier with parameters $(\mathbf{w}_k, b_k)$ and slacks $\boldsymbol{\xi}_k$. Concretely, we solve

$$\begin{aligned} &\min \mathbf{w}_k, b_k, \boldsymbol{\xi}_k \frac{1}{2}\|\mathbf{w}_k\|^2 + C \sum_{i=1}^{n} \xi_{i,k} \\ &\text{subject to} \quad t_{i,k}(\mathbf{w}_k^T \boldsymbol{\phi}(\mathbf{x}_i) + b_k) \geq 1 - \xi_{i,k} \quad \text{for } i = 1, \dots, n \\ &\qquad\qquad \xi_{i,k} \geq 0. \end{aligned}$$

The output of the k-th classifier can be interpreted as a real-valued *score* measuring how confidently $\mathbf{x}$ belongs to class k. Using the dual representation, this score takes the familiar kernel form

$$y_k(\mathbf{x}) = \sum_{i \in \mathcal{S}_k} \lambda_{i,k}^* t_{i,k} \kappa(\mathbf{x}_i, \mathbf{x}) + b_k^*,$$

where $\mathcal{S}_k$ denotes the set of support vectors for the k-th binary problem.

At prediction time, we evaluate all K scores and assign $\mathbf{x}$ to the class with the largest score:

$$\hat{t} = \arg\max k \in \{1, \dots, K\} y_k(\mathbf{x}).$$

In words, OvA chooses the class whose "separate-from-the-rest" hyperplane considers the point most strongly positive.

Advantages and limitations.

The OvA approach is popular because it is simple, modular, and computationally efficient: it requires solving only K binary SVM problems, each of which can be trained with the same algorithms used in the binary case. Moreover, the K classifiers are independent, so one may use different kernels or different regularization parameters C for different classes if the application suggests doing so.

At the same time, the reduction is not without caveats. Since class k typically contains n_k examples, the induced binary dataset for classifier k contains n_k positives and $n - n_k$ negatives; if $n_k \ll n$, the learning problem can be highly imbalanced. In addition, the K hyperplanes are trained independently, so their decision regions are not guaranteed to be globally consistent: it is possible for several classifiers to give similar scores, and (in principle, though rarely in practice) all scores could be negative, making the interpretation of the winning class less geometrically clean than in the binary case. Nevertheless, in many applications OvA performs very well and is often the default choice.

One-vs-One (OvO) alternative.

A widely used alternative is **one-vs-one**. Instead of separating one class from all the others, OvO trains a binary SVM for every pair of classes. For K classes, this yields

$$\binom{K}{2} = \frac{K(K-1)}{2}$$

classifiers. Each classifier is trained only on examples from its two corresponding classes, which typically reduces imbalance and can lead to cleaner binary separations. During prediction, each pairwise classifier casts a vote, and the class with the highest number of votes is selected. The price paid is computational: training and evaluating $\binom{K}{2}$ classifiers can be more expensive than OvA, especially when K is large.

Support Vector Regression

Classification assigns one of finitely many labels to each input point. In many problems, however, the desired output is a real number: predicting prices, temperatures, risk scores, or physical quantities from sensor measurements are typical examples. Support Vector Regression (SVR) extends the SVM framework to this setting while retaining the same core principles: regularize by controlling $\|\mathbf{w}\|$, build a sparse solution in terms of support vectors, and exploit kernel methods to model nonlinear relations.

ϵ-insensitive loss.

The central modeling choice in SVR is the loss function. Instead of requiring a classification margin, SVR introduces an **ϵ-insensitive tube** around the regression function $\hat{y}(\mathbf{x}) = \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}) + b$. Deviations smaller than ϵ are treated as negligible, while larger deviations are penalized linearly. This is formalized by the ϵ-insensitive loss

$$L_\epsilon(y_i, \hat{y}_i) = \begin{cases} 0 & \text{if } |y_i - \hat{y}_i| \leq \epsilon \\ |y_i - \hat{y}_i| - \epsilon & \text{otherwise.} \end{cases}$$

Geometrically, one may picture a tube of width 2ϵ around the prediction function. Points falling inside the tube incur no loss; only points outside the tube contribute to the training penalty. This choice makes the method robust to small perturbations and encourages sparsity, because only points outside (or exactly on) the tube become support vectors in the dual representation.

Primal SVR problem.

To translate this idea into a constrained optimization problem, we introduce two sets of nonnegative slack variables, $\xi_i \geq 0$ and $\xi_i^* \geq 0$, which measure how much the prediction violates the tube above and below, respectively. The constraints enforce that every point lies within the tube up to slack:

$$\begin{aligned} y_i - \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_i) - b &\leq \epsilon + \xi_i \quad \text{for } i = 1, \dots, n, \\ \mathbf{w}^T\boldsymbol{\phi}(\mathbf{x}_i) + b - y_i &\leq \epsilon + \xi_i^* \quad \text{for } i = 1, \dots, n. \end{aligned}$$

The corresponding objective balances flatness of the regression function (controlled by $\|\mathbf{w}\|$) with the total amount of tube violation:

$$
\begin{aligned}
&\min \mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\xi}^* \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} (\xi_i + \xi_i^*) \\
&\text{subject to} \quad y_i - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) - b \leq \epsilon + \xi_i \quad \text{for } i = 1, \dots, n \\
&\qquad\qquad \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b - y_i \leq \epsilon + \xi_i^* \quad \text{for } i = 1, \dots, n \\
&\qquad\qquad \xi_i, \xi_i^* \geq 0.
\end{aligned}
$$

Here, ξ_i quantifies violations of the upper constraint (the true value lies above the tube), while ξ_i^* quantifies violations of the lower constraint (the true value lies below the tube). The parameter C plays the same conceptual role as in soft-margin classification: large C forces the regression function to fit the data more tightly (allowing fewer and smaller violations), while smaller C tolerates larger deviations in exchange for a smoother (lower-norm) predictor.

In the next step, one can proceed exactly as in the classification case: construct the Lagrangian, apply the KKT conditions, and derive a dual problem expressed only through kernel evaluations. The resulting SVR predictor is again a sparse expansion in terms of a subset of training points, reflecting the fact that only points outside the ϵ-tube influence the final regression function.

Computational issues

- Training time of the standard SVM is $O(n^3)$ (solving QP)
  - Can be prohibitive for large datasets
- Lots of research has gone into speeding up the SVMs
  - Many approximate QP solvers are used to speed up SVMs
  - Gradient descent faster and with possibility of limiting the computation time

SVM and Gradient Descent Optimization

The dual formulation of Support Vector Machines provides deep insight into the geometry of the solution and enables the use of kernel methods. At the same time, solving the dual problem requires specialized quadratic programming algorithms, whose computational cost can become prohibitive when the number of training examples is very large. For this reason, it is often advantageous to revisit the primal formulation and ask whether SVMs can be trained using more general-purpose optimization techniques.

An important observation is that, once the constraints are handled appropriately, the soft-margin SVM can be cast as an unconstrained optimization problem. This opens the door to gradient-based methods, including (stochastic) gradient descent, which scale well to large datasets and are widely used in modern machine learning practice.

From the Constrained Primal to an Unconstrained Objective

Recall the primal formulation of the soft-margin SVM:

$$\begin{aligned} \min \mathbf{w}, b, \boldsymbol{\xi} \ & \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \\ \text{subject to } & t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 - \xi_i \quad \text{for } i = 1, \dots, n, \\ & \xi_i \geq 0 \quad \text{for } i = 1, \dots, n. \end{aligned}$$

At first glance, this is a constrained optimization problem involving both the model parameters $(\mathbf{w}, b)$ and the slack variables ξ_i. However, a key simplification arises from the fact that, for fixed $(\mathbf{w}, b)$, the slack variables are completely decoupled from one another and appear only linearly in both the objective and the constraints. As a result, each ξ_i can be optimized independently.

For a given training example $(\mathbf{x}_i, t_i)$, the constraint

$$t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1 - \xi_i$$

forces ξ_i to be at least $1 - t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b)$. Since ξ_i is also constrained to be nonnegative and enters the objective with a positive coefficient, the optimal choice is the smallest value compatible with feasibility. This yields the closed-form expression

$$\xi_i^* = \max \left\{0,\ 1 - t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b)\right\}.$$

Intuitively, if a point already satisfies the margin constraint, no slack is needed. If the constraint is violated, the slack variable measures exactly how far the point lies inside the margin or on the wrong side of the decision boundary.

Substituting these optimal slack values back into the primal objective eliminates the constraints altogether and leads to an unconstrained optimization problem in $(\mathbf{w}, b)$:

$$J(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \max \left\{0,\ 1 - t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b)\right\}.$$

This reformulation makes explicit that training an SVM amounts to minimizing a regularized empirical loss.

Hinge Loss and Regularized Risk Minimization

The quantity inside the summation,

$$\ell_H(\mathbf{w}, b, \mathbf{x}_i, t_i) = \max \left\{0,\ 1 - t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b)\right\},$$

is known as the **hinge loss**. It assigns zero loss to points that are correctly classified with margin at least one, and a positive loss proportional to the amount of margin violation otherwise.

Geometrically, the hinge loss measures how far a training point lies from the margin boundary. Points that are well separated from the decision boundary play no role in shaping the classifier, whereas points that lie inside the margin or are misclassified exert a corrective influence on the parameters.

With this notation, the SVM objective takes the form

$$J(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \ell_H(\mathbf{w}, b, \mathbf{x}_i, t_i),$$

which is a canonical example of a **regularized loss minimization** problem. The first term controls model complexity by penalizing large weights, while the second term enforces fidelity to the training data. The parameter C governs the trade-off between these two goals: increasing C emphasizes correct classification, whereas decreasing C favors wider margins and stronger regularization.

Subgradients of the Hinge Loss

A technical complication arises from the fact that the hinge loss is not differentiable everywhere. In particular, it has a "kink" at the margin boundary where

$$t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) = 1.$$

Nevertheless, the objective function remains convex, and optimization can proceed using the concept of a **subgradient**, which generalizes the notion of a gradient to nonsmooth convex functions.

A vector $\mathbf{g}$ is a subgradient of a convex function f at a point $\mathbf{x}$ if

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{g}^T(\mathbf{y} - \mathbf{x}) \quad \text{for all } \mathbf{y}.$$

For the hinge loss, a valid subgradient with respect to $\mathbf{w}$ is given by

$$\frac{\partial \ell_H}{\partial \mathbf{w}} = \begin{cases} \mathbf{0} & \text{if } t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) \geq 1, \\ -t_i \boldsymbol{\phi}(\mathbf{x}_i) & \text{if } t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) < 1. \end{cases}$$

Aggregating over all training points, the subgradient of the full objective with respect to $\mathbf{w}$ takes the form

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - C \sum_{i \in V} t_i \boldsymbol{\phi}(\mathbf{x}_i),$$

where V denotes the set of indices corresponding to margin-violating examples, that is,

$$V = \left\{ i \mid t_i(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_i) + b) < 1 \right\}.$$

This expression makes clear that only points violating the margin constraint contribute to the data-dependent part of the update.

Gradient Descent Updates

Once the problem has been cast in unconstrained form, standard gradient-based methods can be applied. In batch gradient descent, all parameters are updated simultaneously using the full subgradient at the current iterate:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \left.\frac{\partial J}{\partial \mathbf{w}}\right|_{\mathbf{w}=\mathbf{w}^{(t)}},$$

where $\eta > 0$ is the learning rate.

Substituting the expression for the subgradient yields

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \left(\mathbf{w}^{(t)} - C \sum_{i \in V^{(t)}} t_i \boldsymbol{\phi}(\mathbf{x}_i) \right),$$

or, equivalently,

$$\mathbf{w}^{(t+1)} = (1-\eta)\mathbf{w}^{(t)} + \eta C \sum_{i \in V^{(t)}} t_i \boldsymbol{\phi}(\mathbf{x}_i).$$

This update admits a clear interpretation. The multiplicative factor $(1-\eta)$ shrinks the current weight vector, implementing the effect of regularization and discouraging overly complex models. The additive term pushes the solution in a direction that corrects margin violations, reinforcing the influence of misclassified or weakly classified points. Training examples that are correctly classified with sufficient margin do not affect the update at all.

The bias parameter b can be updated in an analogous way using subgradients, or treated separately depending on the implementation.

Practical Considerations

All other variants of gradient descent, including stochastic and mini-batch methods, can be derived in the same spirit by replacing the full sum over V with estimates based on individual examples or small subsets of the data. These methods dramatically reduce the computational cost per iteration and are the standard choice for large-scale learning problems.

Modern practice. In contemporary machine learning systems, SVMs trained with stochastic or mini-batch gradient descent are often preferred when datasets contain millions of examples or when data arrive in a streaming fashion. Such methods are easy to implement, highly parallelizable, and integrate naturally with modern software frameworks. Dual quadratic programming solvers, on the other hand, remain valuable when exact solutions are required or when the number of training examples is moderate and kernel methods play a central role.

Kernel Methods: Enabling Nonlinear Learning

Up to this point, we have emphasized that many learning algorithms—including linear regression and linear SVMs—are attractive because they are computationally efficient, convex, and relatively easy to analyze. Their main limitation is also clear: a linear decision boundary (or a linear predictor) can only capture linear patterns in

the input variables. When the underlying structure of the data is genuinely nonlinear, linear models can perform poorly even when regularized and carefully tuned.

Kernel methods provide a principled way to overcome this limitation without abandoning the advantages of linear methods. The central idea is to keep the learning algorithm *linear* in a suitable representation of the data, while allowing the *decision boundary in the original input space* to become nonlinear. This is achieved by working in a transformed feature space, but doing so implicitly through inner products, avoiding explicit construction of high-dimensional feature vectors. This mechanism is often referred to as the **kernel trick**.

Motivation: Why Nonlinear Models Matter

Nonlinearity arises naturally in both classification and regression. In classification, the classes may not be separable by any hyperplane in the original space; classic examples include concentric circles or spiral-shaped clusters, where any linear boundary necessarily makes many errors. In regression, the relationship between inputs and outputs may follow a curved trend, include oscillations, or depend on interactions between variables that cannot be captured by a linear function.

A naive approach to nonlinearity is to manually enrich the representation: add polynomial terms, cross-products, trigonometric features, or other nonlinear transformations. While this idea is conceptually simple, it quickly becomes impractical. First, generating a large set of candidate nonlinear features can be computationally expensive. Second, deciding which features to include often requires substantial domain expertise. Third, the dimension of the expanded space can explode, creating serious memory and runtime bottlenecks and increasing the risk of overfitting.

Kernel methods address exactly these issues: they allow us to behave *as if* we had constructed a rich nonlinear feature representation, but without explicitly forming that representation.

Feature Maps and Kernel Functions: Formal Setup

The formal starting point is a **feature map**

$$\boldsymbol{\phi} : \mathcal{X} \to \mathcal{F},$$

where $\mathcal{X}$ is the original input space (typically $\mathbb{R}^d$) and $\mathcal{F}$ is a (possibly very high-dimensional) **feature space**. The space $\mathcal{F}$ is assumed to be a **Hilbert space**, meaning it is a complete vector space equipped with an inner product. Completeness is not merely a technicality: it ensures that optimization and convergence arguments behave well when $\mathcal{F}$ is infinite-dimensional, which is essential for many kernels.

Once $\boldsymbol{\phi}$ is fixed, we apply a linear model in feature space. For SVMs, this means considering decision functions of the form

$$f(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b.$$

The decision boundary is linear in $\mathcal{F}$, but it can correspond to a highly nonlinear surface when viewed back in the original space $\mathcal{X}$.

The difficulty is that explicitly computing $\boldsymbol{\phi}(\mathbf{x})$ may be expensive or even impossible if $\mathcal{F}$ is infinite-dimensional. Kernel methods avoid this by noticing that many learning algorithms—and SVMs in particular—can

be expressed entirely in terms of inner products between feature vectors. This motivates the definition of a **kernel function**

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x}_j).$$

A kernel computes the inner product in feature space while taking as input only the original vectors $\mathbf{x}_i$ and $\mathbf{x}_j$. The feature vectors themselves never need to be constructed. This identity is the essence of the kernel trick: wherever an algorithm requires $\boldsymbol{\phi}(\mathbf{x}_i)^T \boldsymbol{\phi}(\mathbf{x}_j)$, we replace it with $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ and remain entirely in the original input space.

Valid Kernels: Mercer's Condition

Not every function $\kappa(\mathbf{x}, \mathbf{x}')$ can be used as a kernel. For the kernel trick to be meaningful, the function must behave like an inner product in some feature space. The standard characterization is expressed through positive semidefiniteness.

Definition (positive semidefinite kernel). A function $\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a valid kernel if and only if, for every finite set of points $\{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subseteq \mathcal{X}$, the associated **Gram matrix** $\mathbf{K}$ defined by $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite, namely

$$\mathbf{v}^T \mathbf{K} \mathbf{v} \geq 0 \quad \text{for all } \mathbf{v} \in \mathbb{R}^n.$$

Equivalently, all eigenvalues of $\mathbf{K}$ are nonnegative.

This condition guarantees that there exists at least one Hilbert space $\mathcal{F}$ and a feature map $\boldsymbol{\phi}$ such that $\kappa(\mathbf{x}, \mathbf{x}')$ equals the inner product of $\boldsymbol{\phi}(\mathbf{x})$ and $\boldsymbol{\phi}(\mathbf{x}')$. In practical terms, positive semidefiniteness is what makes kernels safe to use: it ensures that the implicit geometry induced by κ is consistent and that optimization problems such as the SVM dual remain well-posed and convex.

Constructing Kernels: Techniques

In practice, one rarely verifies the positive semidefinite condition from scratch for an arbitrary function. Instead, kernels are typically constructed from simpler kernels using operations that preserve validity. If κ_1 and κ_2 are valid kernels, then the following constructions produce valid kernels as well:

1. **Sum:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \kappa_1(\mathbf{x}_1, \mathbf{x}_2) + \kappa_2(\mathbf{x}_1, \mathbf{x}_2)$
2. **Product:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \kappa_1(\mathbf{x}_1, \mathbf{x}_2) \cdot \kappa_2(\mathbf{x}_1, \mathbf{x}_2)$
3. **Scalar multiple:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = c\,\kappa_1(\mathbf{x}_1, \mathbf{x}_2)$ for any $c > 0$
4. **Exponential:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = e^{\kappa_1(\mathbf{x}_1, \mathbf{x}_2)}$
5. **Polynomial:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = p(\kappa_1(\mathbf{x}_1, \mathbf{x}_2))$ for any polynomial p with nonnegative coefficients
6. **Weighted inner product:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{A} \mathbf{x}_2$ for any positive definite matrix $\mathbf{A}$
7. **Scaling:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = f(\mathbf{x}_1)\kappa_1(\mathbf{x}_1, \mathbf{x}_2)g(\mathbf{x}_2)$ for any functions $f, g : \mathcal{X} \to \mathbb{R}$
8. **Feature map composition:** $\kappa(\mathbf{x}_1, \mathbf{x}_2) = \kappa_3(\boldsymbol{\phi}(\mathbf{x}_1), \boldsymbol{\phi}(\mathbf{x}_2))$ where $\boldsymbol{\phi}$ is any feature map and κ_3 is any kernel on the transformed space

These closure properties explain why kernel design is flexible: complex similarity measures can be engineered by combining simpler building blocks, all while guaranteeing that the resulting kernel corresponds to some valid feature space geometry.

Polynomial Kernels: Example Construction

A canonical example is the polynomial kernel

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + c)^d.$$

This kernel can be justified constructively. The basic dot product $\mathbf{x}_1^T \mathbf{x}_2$ is itself a kernel, corresponding to the identity feature map. A constant kernel c is also valid, corresponding to adding a constant feature. Their sum $\mathbf{x}_1^T \mathbf{x}_2 + c$ is therefore a kernel. Finally, raising a kernel to an integer power d (equivalently applying the polynomial $p(z) = z^d$ with nonnegative coefficients) produces another valid kernel.

To make the feature expansion explicit, consider $d = 2$ in $\mathbb{R}^2$. We obtain

$$\begin{aligned}\kappa(\mathbf{x}_1, \mathbf{x}_2) &= (x_{1,1}x_{2,1} + x_{1,2}x_{2,2} + c)^2 \\ &= \sum_{i,j} x_{1,i}x_{1,j}x_{2,i}x_{2,j} + 2c \sum_i x_{1,i}x_{2,i} + c^2.\end{aligned}$$

This corresponds to the feature map

$$\boldsymbol{\phi}(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2,\ \sqrt{2c}x_1,\ \sqrt{2c}x_2,\ c)^T,$$

which maps from $\mathbb{R}^2$ to $\mathbb{R}^6$.

The computational point is crucial. Even though the feature dimension grows rapidly with the polynomial degree, evaluating $\kappa(\mathbf{x}_1, \mathbf{x}_2)$ still requires only $O(d)$ work: a single dot product followed by a scalar power. In contrast, explicitly constructing the polynomial feature vector of degree t would require a number of components on the order of $O(d^t)$. The kernel trick therefore provides a dramatic savings in computation and memory.

Gaussian Kernel: Infinite-Dimensional Features

Among the most widely used kernels is the **Gaussian** (or RBF) kernel

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\frac{\|\mathbf{x}_1 - \mathbf{x}_2\|^2}{2\sigma^2}\right),$$

where $\sigma > 0$ is a bandwidth parameter.

To see why this function is a valid kernel, one can start from the identity

$$\|\mathbf{x}_1 - \mathbf{x}_2\|^2 = \|\mathbf{x}_1\|^2 + \|\mathbf{x}_2\|^2 - 2\mathbf{x}_1^T \mathbf{x}_2$$

and rewrite the exponential as

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = e^{-\|\mathbf{x}_1\|^2/(2\sigma^2)} \cdot e^{\mathbf{x}_1^T\mathbf{x}_2/\sigma^2} \cdot e^{-\|\mathbf{x}_2\|^2/(2\sigma^2)}.$$

The middle factor is the exponential of a scaled linear kernel and defines a valid kernel, while the outer factors depend only on $\mathbf{x}_1$ and $\mathbf{x}_2$ separately and can be seen as scaling terms; scaling a valid kernel in this way preserves validity. This decomposition provides an explicit route to verifying that the Gaussian kernel satisfies the kernel conditions.

The feature-space interpretation is equally important: the Gaussian kernel corresponds to an infinite-dimensional feature map. Although $\boldsymbol{\phi}(\mathbf{x})$ has infinitely many components, the kernel trick allows inner products to be computed exactly with a simple closed-form expression. This is the source of the Gaussian kernel's remarkable flexibility: it can represent extremely rich families of decision boundaries while keeping computations manageable through kernel evaluations.

The bandwidth σ controls the locality of the similarity measure. When σ is small, $\kappa(\mathbf{x}_1, \mathbf{x}_2)$ is close to 1 only for very nearby points, which allows highly flexible, finely detailed decision boundaries but increases the risk of overfitting. When σ is large, the kernel becomes smoother and more global, which can improve robustness but may lead to underfitting. As with C in soft-margin SVMs, σ is typically selected by cross-validation.

Common Kernel Functions and When to Use Them

Different kernels encode different inductive biases, meaning different assumptions about what kinds of patterns are likely to appear in the data. The most common choices include:

Linear kernel. The simplest kernel is

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T\mathbf{x}_2.$$

It is appropriate when the data are already (approximately) linearly separable and is the fastest and least prone to overfitting. It also preserves interpretability in the original feature space.

Polynomial kernel. A standard nonlinear alternative is

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T\mathbf{x}_2 + c)^d.$$

It introduces controlled nonlinearity: the degree d governs complexity, while c adjusts the relative weight of lower- and higher-order terms. Polynomial kernels are often effective when interactions between features matter but extremely localized boundaries are not needed.

Gaussian (RBF) kernel. The Gaussian kernel can also be written as

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = \exp(-\gamma\|\mathbf{x}_1 - \mathbf{x}_2\|^2),$$

where $\gamma = 1/(2\sigma^2)$. It is a strong default choice when one expects complex nonlinear structure and has enough data to tune hyperparameters and avoid overfitting. Its performance can degrade in very high dimensions due to distance concentration effects.

Sigmoid kernel. Another option is

$$\kappa(\mathbf{x}_1, \mathbf{x}_2) = \tanh(c_1 \mathbf{x}_1^T \mathbf{x}_2 + c_2).$$

It is historically motivated by neural networks, but it does not satisfy the kernel validity condition for all parameter choices, so it should be used with care.

Kernels in SVM: The Full Picture

We can now recall and summarize how kernels enter the SVM training and prediction pipeline. In the soft-margin case, the dual optimization problem takes the kernelized form

$$\max \boldsymbol{\lambda} \left(\sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j t_i t_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \right)$$

$$\text{subject to} \quad 0 \le \lambda_i \le C \quad \text{for } i = 1, \dots, n$$

$$\sum_{i=1}^{n} \lambda_i t_i = 0.$$

Once the optimal multipliers $\boldsymbol{\lambda}^*$ are found, prediction for a new point $\mathbf{x}$ uses the decision function

$$y(\mathbf{x}) = \sum_{i \in \mathcal{S}} \lambda_i^* t_i \kappa(\mathbf{x}_i, \mathbf{x}) + b^*,$$

where $\mathcal{S}$ is the set of support vectors.

This representation makes clear why kernelized SVMs are powerful. Nonlinearity is entirely contained in the kernel κ, while the learning algorithm remains a convex optimization problem with strong theoretical guarantees. At the same time, sparsity is preserved: only support vectors contribute to predictions, which keeps evaluation efficient. Finally, kernel choice provides a flexible way to inject prior knowledge about similarity and structure into the learning process, enabling SVMs to adapt to a wide variety of datasets without changing the underlying optimization framework.